

Multi-Agent Intelligence Hub

Python Practical Submission

Name: Sneha Gautam
Roll No: A26MTAIN0019

Submission Note

The shutdown triggered flag is used to remember whether a STOP signal has appeared inside the inner loop. At the beginning, the flag is set to False, which means that no shutdown has been detected. When the loop reads a STOP signal, it changes the flag to True and uses break to stop the inner loop. The outer loop can then check this flag to know whether an emergency shutdown occurred. The outer loop's else clause is executed only when the loop finishes normally without encountering a break. Therefore, when STOP is detected, the flag becomes True and the program can handle the shutdown condition instead of treating the loop as a normal completion. If no STOP signal is found, the flag remains False, so the outer loop can complete normally and its else clause can execute. In this way, the shutdown flag clearly records whether a STOP signal was ever encountered during processing.

Python Code

```
1  # Multi-Agent Intelligence Hub
2
3  agent_registry = {
4      "Agent_Alpha": {
5          "Location": (10, 20),
6          "Knowledge": {"grid_map", "comm_protocol", "
              telemetry_sync"}
7      },
8
9      "Agent_Beta": {
10         "Location": (15, 5),
11         "Knowledge": {"grid_map", "comm_protocol", "power_grid"}
12     },
13
14     "Agent_Gamma": {
15         "Location": (0, 0),
16         "Knowledge": {"grid_map", "comm_protocol", "power_grid",
              "telemetry_sync"}
17     }
18 }
19
20
21 # Find knowledge common to all agents
22 def find_common_intelligence(registry):
```

```

23     knowledge_sets = []
24
25     for data in registry.values():
26         knowledge_sets.append(data["Knowledge"])
27
28     return set.intersection(*knowledge_sets)
29
30
31 common = find_common_intelligence(agent_registry)
32
33 print("--- Common Intelligence Across All Agents ---")
34 print(common)
35
36
37 # Movement log
38 movement_log = {}
39
40
41 def relocate_agent(registry, log, agent_id, new_location):
42
43     old_location = registry[agent_id]["Location"]
44
45     print("Trying to change the old location tuple...")
46
47     try:
48         old_location[0] = new_location[0]
49
50     except TypeError:
51         print("TypeError caught!")
52         print("Tuple cannot be changed, so we will create a new
53             tuple.")
54
55     # Change complete Location value
56     registry[agent_id]["Location"] = new_location
57
58     # Store movement
59     log.setdefault(agent_id, [])
60     log[agent_id].append(new_location)
61
62     print("New state:", registry[agent_id])
63     print("Movement Log:", log)
64
65 print("\n--- Relocating Agent_Alpha ---")
66
67 relocate_agent(
68     agent_registry,
69     movement_log,
70     "Agent_Alpha",
71     (12, 22)
72 )

```

```

73
74
75 print("\n--- Relocating Agent_Alpha Again ---")
76
77 relocate_agent(
78     agent_registry,
79     movement_log,
80     "Agent_Alpha",
81     (14, 25)
82 )
83
84
85 # Summary report
86 summary_report = {
87     agent_id: len(data["Knowledge"])
88     for agent_id, data in agent_registry.items()
89 }
90
91 print("\n--- Summary Report ---")
92 print(summary_report)

```

Program Output

The following is the output obtained after running the Python program.

```

>>> = RESTART: C:/Users/Sneha Gautam/Downloads/COLLEGE/AI/Assignment_7_lab/Multi-Agent Intel
ligence Hub.py
--- Common Intelligence Across All Agents ---
{'comm_protocol', 'grid_map'}

--- Relocating Agent_Alpha ---
Trying to change the old location tuple...
TypeError caught!
Tuple cannot be changed, so we will create a new tuple.
New state: {'Location': (12, 22), 'Knowledge': {'comm_protocol', 'grid_map', 'telemetry_sync'}}
Movement Log: {'Agent_Alpha': [(12, 22)]}

--- Relocating Agent_Alpha Again ---
Trying to change the old location tuple...
TypeError caught!
Tuple cannot be changed, so we will create a new tuple.
New state: {'Location': (14, 25), 'Knowledge': {'comm_protocol', 'grid_map', 'telemetry_sync'}}
Movement Log: {'Agent_Alpha': [(12, 22), (14, 25)]}

--- Summary Report ---
{'Agent_Alpha': 3, 'Agent_Beta': 3, 'Agent_Gamma': 4}
>>>

```

Figure 1: Program Output

Brief Explanation

The program maintains information about three agents using a dictionary. Each agent has a location stored as a tuple and a set containing its knowledge. The common knowledge is found using set intersection. During relocation, the program attempts to change an element of the old location tuple. This produces a **TypeError** because tuples are immutable. The error is caught using **try-except**, after which the complete location is replaced with a new tuple. The movement is stored in a dictionary using **setdefault()** and **append()**. Finally, a dictionary comprehension creates the summary report by counting the knowledge items of each agent.

Note

The current uploaded program demonstrates dictionaries, sets, tuples, functions, exception handling, **setdefault()**, lists, and dictionary comprehension. The displayed program does not itself contain a **STOP** signal or shutdown flag; therefore, the shutdown note above explains the required flag mechanism separately.